Real Estate Property Management Platform

INTRODUCTION:

RealEst is a versatile and user-friendly software platform designed to streamline the creation and management of rental properties, tenant records, and payment operations. Suitable for property owners, managers, real estate agencies, and tenant management, it offers a wide range of customizable property features—including property listings with detailed descriptions, tenant databases, lease management, and payment tracking—that can be organized through an intuitive drag-and-drop dashboard interface. Users can fully tailor their property portfolios and operational workflows to match their business needs, while robust security features like role-based access control (RBAC), JWT authentication, SSL encryption, and submission verification help protect sensitive data integrity. RealEst also integrates seamlessly with Razorpay for secure payment processing and Google Maps for location-based property discovery, automating workflows and payment notifications. With built-in data visualization tools and advanced analytics, users can generate comprehensive reports on occupancy rates, payment collections, and rental trends, with the ability to export responses for deeper analysis. Its flexibility, ease of use, powerful automation features, and enterprise-grade security make it an ideal choice for property owners, real estate investment firms, property management companies, housing institutions, and property seekers seeking an efficient property management solution.

SCENARIO-BASED CASE STUDY:

Meet Priya - A Property Manager's Journey with RealEst

Priya is an experienced property manager overseeing a diverse portfolio of 15 residential and commercial properties across the city. With a busy schedule managing multiple properties, tenant interactions, lease agreements, and payment collections, Priya struggles with scattered spreadsheets, missed payment reminders, and time-consuming manual processes. She needs a comprehensive solution to centralize her operations and improve efficiency.

Priya discovers RealEstate, a modern property management platform designed to simplify real estate operations. Intrigued by its potential, Priya decides to explore how RealEst can transform her daily workflows.

User Registration and Authentication: Priya registers an account on RealEst, providing her professional credentials and role details as a property manager. She logs in securely using her username and password, ensuring her sensitive property and financial data remains private and encrypted. The platform's JWT authentication and SSL encryption give her peace of mind about data security.

Property Portfolio Management: Priya explores RealEst's intuitive dashboard and begins uploading her 15 properties. For each property, she adds comprehensive details—location, monthly rent, amenities, property type, and high-quality images. Using the drag-and-drop interface, she organizes properties by location and type. Google Maps integration helps her visualize property locations geographically, making it easy for potential tenants to discover properties.

Tenant Management System: RealEst's tenant database allows Priya to maintain detailed profiles for all her tenants. She can track tenant information, lease start/end dates, contact details, and payment history in one centralized location. When new rental requests come in from prospective tenants, Priya reviews them through the Request Management system and accepts or rejects applications seamlessly.

Digital Lease Management: Instead of managing dozens of paper lease agreements, Priya now creates and stores digital lease documents directly in RealEst. When a tenant is approved, the lease agreement is automatically generated, stored securely, and linked to both the property and tenant profile. This eliminates paperwork clutter and enables quick access to any agreement whenever needed.

Payment Tracking and Collections: RealEst's integrated Razorpay payment gateway transforms Priya's rent collection process. Tenants can now pay their monthly rent securely through the platform, and payments are automatically recorded. Priya receives instant notifications of successful payments and can easily track which tenants have paid and which are pending. The system automatically updates payment statuses, giving her real-time visibility into cash flow.

Automated Reminders and Notifications: Priya sets up automated cron jobs through RealEst that send payment reminders to tenants before their rent is due. When a lease is approaching its expiration date, the system sends notifications to both Priya and the tenant, ensuring smooth transitions and renewals. This automation reduces the burden of manual follow-ups.

Sales Reporting and Analytics: RealEst provides Priya with powerful analytics tools. She can generate comprehensive reports on:

- Monthly rental income and collections
- Occupancy rates across her portfolio
- Pending vs. completed payments
- Tenant demographics and distribution
- Lease expiration timelines

These insights help Priya identify trends, forecast revenue, and make data-driven decisions about her property portfolio.

Tenant-Manager Communication: Through RealEst's integrated request and messaging system, Priya can easily communicate with tenants about maintenance issues, lease renewals, and payment queries. Tenants can submit requests or inquiries directly through the platform, creating a transparent and organized communication channel that reduces misunderstandings.

Training and Support: RealEst provides Priya with comprehensive training materials, video tutorials, and responsive customer support. She can access FAQs about property management workflows, payment processing, and troubleshooting. The support team helps her resolve any technical issues quickly, ensuring minimal disruption to her operations.

Priya's Experience and Results: Within three months of using RealEst, Priya experiences significant improvements:

- 70% reduction in administrative time - Automation and centralization eliminate manual data entry and spreadsheet management
- 100% on-time payment tracking - Automated reminders and digital payment processing reduce delinquency from 15% to 2%
- Improved tenant satisfaction - Transparent communication and quick response times enhance tenant relationships
- Better decision-making - Analytics and reports provide insights for portfolio optimization
- Scalability - Managing 15 properties becomes effortless, and Priya confidently plans to expand her portfolio to 25 properties

- Data security - Peace of mind knowing sensitive property and financial data is encrypted and secure

Priya's Conclusion: "RealEstate has transformed my property management business. What used to take hours now takes minutes. I can focus on growing my portfolio and building better relationships with my tenants instead of getting lost in administrative work. RealEst has become an indispensable tool that has helped me achieve my professional goals with ease and confidence."

SYSTEM REQUIREMENTS:

1. Software Requirements:

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- Operating System: Windows 10/11, macOS, or Linux - Supports cross-platform development and testing.
- Node.js (v16 or above): Provides the runtime environment for building frontend logic and powers the server-side logic and handles API routing. [Download Node.js](https://nodejs.org/)
- npm (v8 or above): A package manager required to install dependencies for React, Express, and related libraries.
- React.js: A JavaScript library for building dynamic and responsive user interfaces for the frontend.
- Express.js: A lightweight web framework for building RESTful APIs and handling backend server operations.
- MongoDB: A NoSQL database used to store structured and unstructured data related to properties, users, tenants, leases, and payments. [Download MongoDB](https://www.mongodb.com/try/download/community)
- Browser: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time during development and usage.
- Razorpay Account: Required for payment gateway integration and online payment processing. [Create Razorpay Account](https://razorpay.com/)
- Postman: Tool for testing APIs during development and troubleshooting backend endpoints.
- Visual Studio Code: Preferred code editor with built-in Git and terminal support for efficient development.
- Git: Version control system for managing code repositories and collaboration.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- RAM: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, MongoDB, and browser testing simultaneously.
- Storage: At least 1 GB free space — Required for package installations, MongoDB setup, project files, and property data/documents storage.
- Display: 1366x768 or higher — Recommended for optimal coding experience and property management interface visualization.
- Internet Connection: Stable broadband connection required for API calls, payment processing, and cloud database operations.

All software components are freely available and open-source, making RealEst cost-effective to develop and deploy.

TECHNICAL ARCHITECTURE:

The RealEst platform follows a layered, microservices-inspired architecture designed for scalability, maintainability, and performance. The system is structured with clear separation of concerns across multiple layers:

1. User Interface Layer

- **React.js Frontend Application** - Interactive user interface for property owners, managers, and tenants
- **Responsive Design** - Works on desktop, tablet, and mobile devices
- **User Roles** - Different interfaces based on user role (owner, manager, tenant)

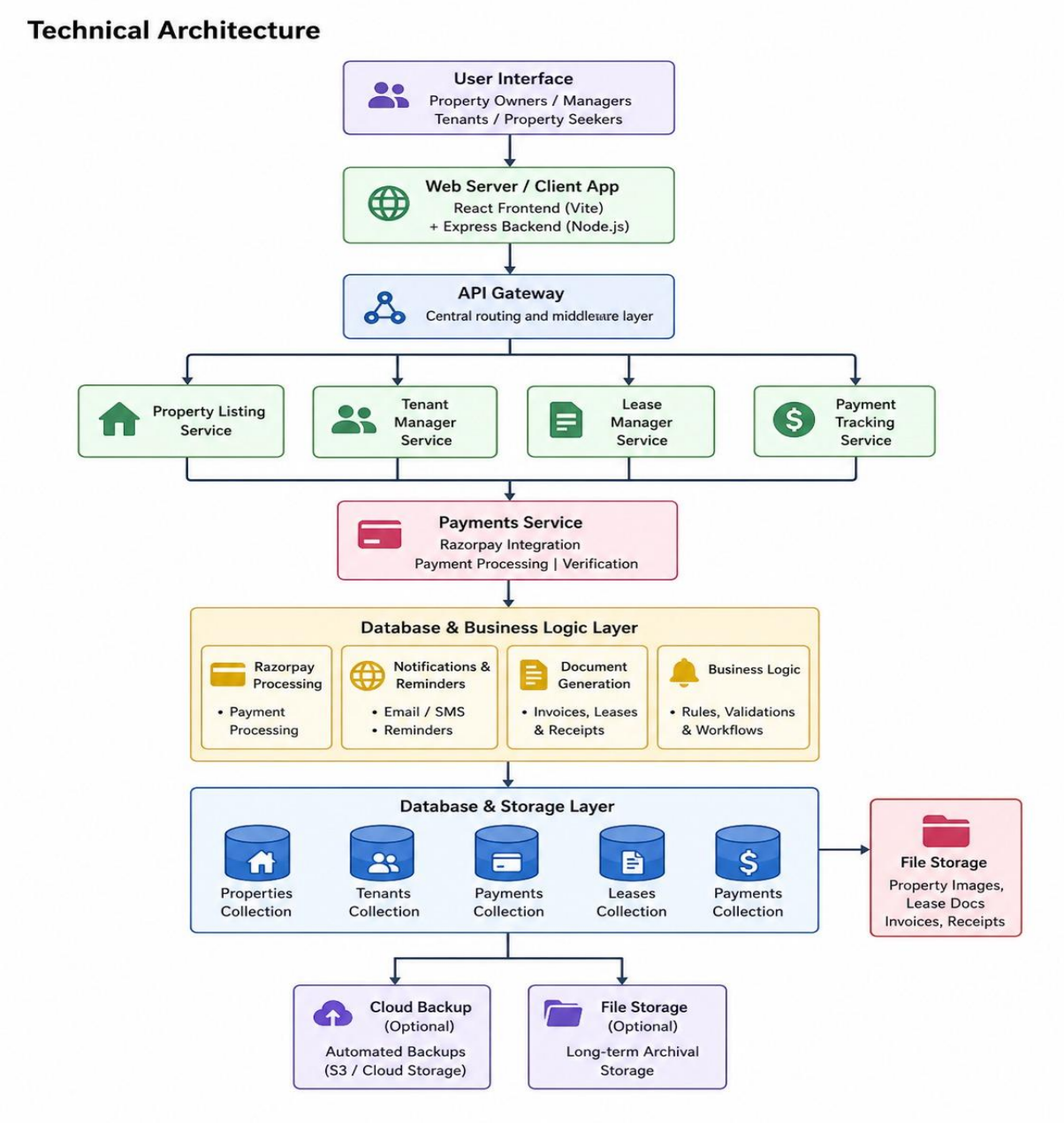
2. Web Server / Client Application Layer

- **Frontend**: Vite + React.js bundled application
- **Backend**: Express.js REST API server
- **Communication**: RESTful APIs via HTTP/HTTPS with JWT token-based authentication

3. API Gateway Layer

- **Central Entry Point** - Routes all requests to appropriate services
- **Middleware Stack**:
    - Request validation

- Authentication (JWT verification)
- Authorization (role-based access control)
- Error handling
- CORS management
- Request logging



Core Microservices:

Property Listings Service

- Create, read, update, delete properties
- Property search and filtering
- Manage property metadata (location, rent, amenities)
- Property image management

Tenant Management Service

- Maintain tenant profiles and records
- Track tenant rental history
- Manage tenant-property associations
- Handle request statuses (pending, accepted, rejected)

Lease Management Service

- Create and manage lease agreements
- Store digital lease documents
- Track lease timelines (start/end dates)
- Monitor lease status (active/completed)

Payment Tracking Service

- Record rental payments
- Track payment statuses
- Maintain transaction history
- Generate payment reports

Payments Service Layer

- **Razorpay Integration**: Secure payment gateway for online rent collection
- **Payment Processing**: Create orders, verify signatures, handle callbacks
- **Transaction Management**: Record and track all payment transactions
- **Security**: SSL encryption for sensitive payment data

Business Logic & Integration Layer

- **Razorpay Payment Processing**: Handles payment creation, verification, and reconciliation
- **Notifications & Reminders**: Automated cron jobs for payment reminders and lease expiry alerts
- **Document Generation**: Create and manage lease documents, generate PDFs

Database & Storage Layer

MongoDB Collections:

- **Properties**: Property listings with details, location, rent information
- **Tenants/Users**: User profiles with roles and authentication data
- **Payments**: Transaction records with status and timestamps
- **Leases**: Lease agreements with start/end dates and document links

File Storage:

- **Property Images**: Uploaded property photos
- **Lease Documents**: Digital lease agreement PDFs
- **User Uploads**: Tenant documents and supporting files

Key Technologies & Services

Layer	Technology	Purpose
Frontend	React.js, Vite, TailwindCSS	User interface & styling
Backend	Node.js, Express.js	API & server logic
Database	MongoDB, Mongoose	Data persistence
Authentication	JWT, bcryptjs	Secure authentication
Payment Gateway	Razorpay	Online payment processing
File Handling	Multer, Node.js fs	File upload & storage
Scheduling	node-cron	Automated tasks
Maps	Google Maps API	Location visualization
Notifications	Email/SMS via Razorpay	Automated alerts

Data Flow:

1. **User Interaction**: Property owner/tenant interacts with React frontend
2. **API Request**: Frontend sends HTTP request with JWT token to API Gateway
3. **Authentication**: API Gateway validates JWT and authorization
4. **Service Processing**: Request routed to appropriate microservice
5. **Database Query**: Service queries MongoDB for required data
6. **Business Logic**: Processing includes payment verification, lease validation, etc.
7. **Response**: Service returns data back through API Gateway to frontend
8. **UI Update**: Frontend updates UI with received data

Security Architecture:

- **Authentication**: JWT tokens for stateless authentication
- **Authorization**: Role-based access control (RBAC) for different user types
- **Encryption**: SSL/TLS for data in transit
- **Password Security**: bcryptjs hashing for stored passwords
- **API Validation**: Input validation on all endpoints

- **CORS**: Configured for secure cross-origin requests
- **File Upload**: Validated file types and size restrictions

Scalability Considerations:

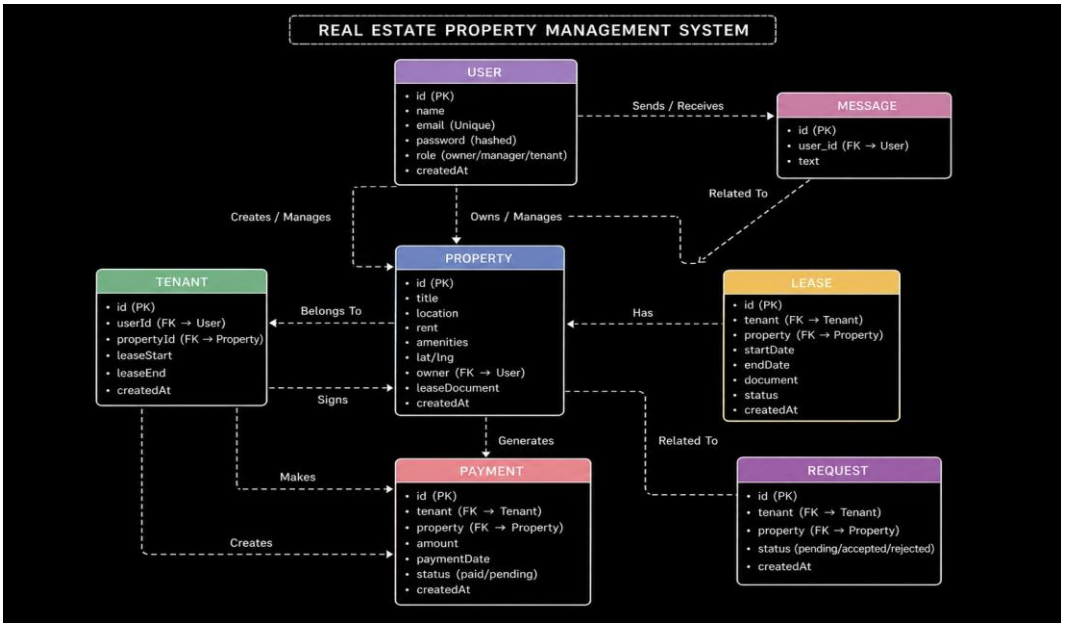
- **Microservices Design:** Services can be scaled independently
- **Database Indexing**: MongoDB indexes on frequently queried fields
- **Caching**: Can implement Redis for frequently accessed data
- **Load Balancing**: API Gateway can be replicated behind load balancer
- **Horizontal Scaling**: Stateless design allows multiple backend instances
- **Cloud Deployment**: Architecture supports AWS, Azure, or Google Cloud deployment

This architecture provides a robust, secure, and scalable foundation for the RealEst property management platform, enabling efficient handling of multiple users, properties, and transactions while maintaining data integrity and system reliability.

ER DIAGRAM (Entity-Relationship Diagram):

The Entity-Relationship (ER) diagram for RealEst represents the various entities and their relationships within the property management system. Here is a description of the key components of the ER diagram:

Visual Representation



The Entity-Relationship (ER) diagram for the **Real Estate Rental Management System** represents the various entities and their relationships within the property rental management platform. Here is a description of the key components of the ER diagram:

1. Entities:

●User:

Represents all users in the system, including property owners, managers, and tenants, with attributes such as user ID, name, email, password, role, phone number, and timestamps.

●Property:

Represents rental properties available in the system, with attributes such as property ID, title, location, rent amount, amenities, map coordinates, lease documents, and owner details.

●Tenant:

Represents tenant records and their assigned properties, with attributes such as tenant ID, lease start date, lease end date, occupancy status, and property assignment details.

●Lease:

Represents lease agreements between tenants and property owners, with attributes such as lease ID, start date, end date, lease document, and lease status.

●Payment:

Represents rental payment transactions made by tenants, with attributes such as payment ID, amount, payment date, status, and payment transaction details.

●Request:

Represents rental requests submitted by tenants for properties, with attributes such as request ID, request status, tenant message, and timestamps.

2. Relationships:

●UserProperty:

Represents the relationship between users and properties, indicating that one property owner can own multiple properties.

●UserTenant:

Represents the relationship between users and tenant records, indicating that one user can have one tenant profile.

●PropertyLease:

Represents the relationship between properties and lease agreements, indicating that one property can have multiple lease agreements.

●UserLease:

Represents the relationship between tenants and lease agreements, indicating that one tenant can have multiple lease records.

●LeasePayment:

Represents the relationship between lease agreements and payment records, indicating that one lease can have multiple payment transactions.

●PropertyPayment:

Represents the relationship between properties and payment records, indicating that one property can have multiple payment transactions.

●UserPayment:

Represents the relationship between tenants and payment records, indicating that one tenant can make multiple rent payments.

●UserRequest:

Represents the relationship between tenants and rental requests, indicating that one tenant can submit multiple requests.

●PropertyRequest:

Represents the relationship between properties and rental requests, indicating that one property can receive multiple requests.

3. Attributes

Each entity has its own set of attributes that describe its properties.

For example:

- The **User** entity contains attributes such as user ID, name, email, password, role, and phone number.
- The **Property** entity contains attributes such as property ID, title, location, rent, and amenities.
- The **Tenant** entity contains lease duration and occupancy status.
- The **Lease** entity contains start date, end date, and lease document details.
- The **Payment** entity contains payment amount, payment date, and status.
- The **Request** entity contains request status and tenant messages.

4. Primary Keys:

Each entity has a primary key that uniquely identifies each record in the entity.

For example:

- The **User** entity's primary key is the **User ID**.
- The **Property** entity's primary key is the **Property ID**.
- The **Tenant** entity's primary key is the **Tenant ID**.

- The **Lease** entity's primary key is the **Lease ID**.
- The **Payment** entity's primary key is the **Payment ID**.
- The **Request** entity's primary key is the **Request ID**.

5. Foreign Keys:

Foreign keys are used to establish relationships between entities.

For example:

- The **Property** entity has a foreign key referencing the **User ID** to link properties with owners.
- The **Tenant** entity has foreign keys referencing **User ID** and **Property ID** to link tenants with user accounts and properties.
- The **Lease** entity has foreign keys referencing **User ID** and **Property ID** to connect lease agreements with tenants and properties.
- The **Payment** entity has foreign keys referencing **User ID** and **Property ID** to record tenant payments.
- The **Request** entity has foreign keys referencing **User ID** and **Property ID** to manage rental requests.

KEY FEATURES

The key features of RealEst property management platform are designed to streamline operations for property owners, managers, and tenants:

1. User Registration and Profiles

- Easy account creation with role-based setup (owner/manager/tenant)
- Secure authentication using JWT tokens and bcryptjs password hashing
- Personal profile management with editable information and document uploads
- Profile verification and account recovery functionality

2. Property Listing and Management

- Create and manage property listings with comprehensive details (title, location, rent, amenities)
- Upload property images and set availability status
- Drag-and-drop interface for easy property organization
- Integration with Google Maps for location visualization
- Bulk management for multiple properties

3. Rental Request Management

- Tenants can browse properties and submit formal rental requests
- Attach supporting documents (ID, income proof, references)
- Owners/managers can accept or reject requests with custom messages
- Track request status in real-time (pending, accepted, rejected)

4. Digital Lease Management

- Automatically generate customized lease documents
- Upload and store digital lease templates for standardization
- Electronic signature support for lease agreements
- Version control and automatic expiration reminders
- Complete lease history for audit purposes

5. Payment Integration and Tracking

- Razorpay payment gateway integration for secure rent collection
- Multiple payment methods (credit/debit cards, wallets, UPI)
- Recurring payment setup for automatic monthly collection
- Real-time payment status monitoring and analytics
- Payment reports, invoices, and receipts

6. Booking History and Records

- Maintain comprehensive records of past and upcoming bookings
- Track rental duration, amounts, and payment histories
- Store lease agreements and tenant references
- Property maintenance and repair history tracking
- Generate historical reports and analytics

7. Automated Notifications and Reminders

- Payment reminders sent before due dates
- Lease expiration notifications (30, 15, 7 days before)
- Late payment alerts and maintenance reminders
- Customizable alert preferences and notification frequency

8. Analytics and Reporting

- Dashboard with rental income and collection status
- Occupancy rate analysis across properties

- Payment trends and revenue forecasting
- Export reports in PDF, Excel, and CSV formats
- Customizable reports based on specific metrics

9. Tenant-Manager Communication
- Integrated messaging system for direct communication
- Maintenance request submissions with photo attachments
- Organized message history and conversation threads
- Message status tracking and request follow-ups

10. Role-Based Access Control
- Different access levels for owners, managers, and tenants
- Secure permission system with activity logs
- Encryption of sensitive information
- Compliance with data protection regulations

11. Mobile and Desktop Accessibility
- Responsive design works on all devices (mobile, tablet, desktop)
- Consistent user experience across all platforms
- Cloud synchronization for seamless data access

12. Customer Support
- Comprehensive FAQs and knowledge base
- Video tutorials for common tasks
- Live customer support via chat and email
- Ticketing system for issue tracking
- Regular webinars and training sessions

ROLES AND RESPONSIBILITIES:

Property Owner:

● **Property Registration and Management:** Property owners are responsible for listing their rental properties on the platform, providing accurate property details such as location, rent, amenities, and availability, and updating the information when required.

● **Tenant Request Management:** Property owners review tenant rental requests, verify tenant details, and make decisions to accept or reject rental applications.

● **Lease Management:** Property owners are responsible for creating and maintaining lease agreements, uploading lease documents, and managing lease renewals or terminations.

● **Payment Monitoring:** Property owners track rental payments, monitor pending and completed transactions, and ensure timely rent collection.
● **Communication:** Property owners communicate with tenants and property managers regarding lease terms, rental issues, and maintenance concerns.
● **Reports and Analytics:** Property owners can access revenue reports, occupancy details, and payment analytics to monitor property performance.

Property Manager:
● **Property Oversight:** Property managers are responsible for managing assigned properties on behalf of property owners and maintaining updated property records.
● **Tenant Communication:** Property managers handle tenant inquiries, maintenance requests, and general communication related to the rental process.
● **Request Processing:** Property managers review and process rental requests, including approving or rejecting applications if authorized.
● **Lease Handling:** Property managers assist in creating, updating, and maintaining lease agreements and related documents.
● **Payment Tracking:** Property managers monitor payment statuses, send reminders, and manage overdue payment notifications.
● **Maintenance Coordination:** Property managers handle maintenance records and coordinate repair requests for properties.

Tenant:
● **Registration and Account Management:** Tenants are responsible for creating accounts, maintaining profile details, and ensuring secure account credentials.
● **Property Search and Selection:** Tenants can browse available rental properties based on preferences such as location, rent, and amenities.
● **Rental Request Submission:** Tenants are responsible for submitting rental requests and providing necessary supporting documents for verification.
● **Lease Review and Agreement:** Tenants review lease agreements, understand the terms, and sign lease documents when approved.
● **Rent Payment:** Tenants are responsible for making rent payments through available payment methods and maintaining payment records.
● **Communication:** Tenants can communicate with property owners or managers regarding rental issues, lease-related questions, or maintenance requests.
● **Feedback and Updates:** Tenants can provide feedback and keep their personal information updated in the system.

User Flow:

Property Owner Flow

● **Registration & Account Setup** – Create an account and log in to the platform.

● **Property Listing** – Add new properties with complete details, images, and rental terms.

● **Rental Request Review** – Check incoming rental requests from tenants and review tenant details.

● **Lease Creation** – Approve requests and generate lease agreements for selected tenants.

● **Payment Monitoring** – Track tenant payments and payment status.

● **Reports & Analytics** – View rental income, occupancy rates, and financial reports.

Property Manager Flow

● **Login & Property Access** – Log in and access assigned property details.

● **Request Management** – Review and process tenant rental requests.

● **Tenant Communication** – Manage tenant queries, maintenance requests, and support.

● **Lease Management** – Create and update lease agreements.

● **Payment Tracking** – Monitor payments and send payment reminders.

● **Maintenance Coordination** – Handle property maintenance and service records.

Tenant Flow

● **Registration & Account Setup** – Create an account and manage profile details.

● **Property Search** – Browse and search available properties using filters.

● **Rental Request Submission** – Select a property and submit a rental request.

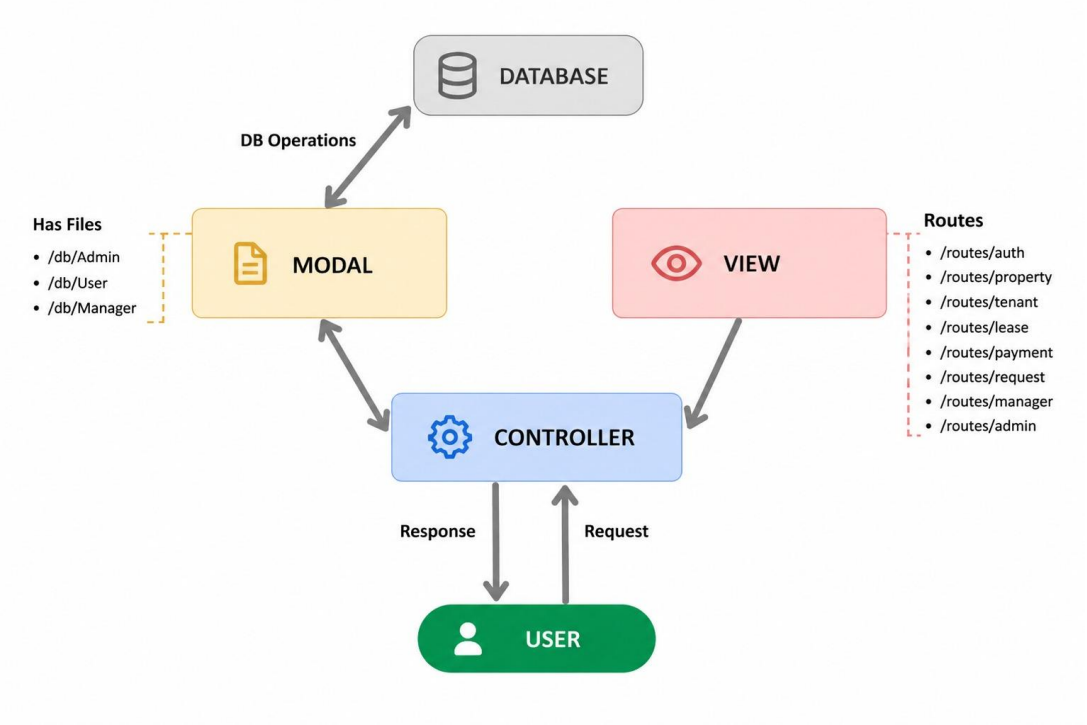
● **Lease Agreement** – Review and sign the lease document after approval.

● **Payment Process** – Make rental payments and receive receipts.

● **Communication** – Contact owners or managers for support and maintenance issues.

● **Notifications** – Receive reminders for payments, lease expiry, and request updates.

MVC PATTERN :



The Real Estate Rental Management System follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates the application into three major layers: Model, View, and Controller. This architecture helps in organizing the application efficiently, making it easier to manage, maintain, and scale as the project grows.

Model Layer (Data Layer)

The **Model layer** is responsible for managing the application's data and database operations. It defines the data structure, relationships, and constraints for the system using **Mongoose schemas** with **MongoDB** as the database.

The models in this project include:

- **User Model** → Stores user information such as name, email, password, role, and phone number.
- **Property Model** → Stores property details such as title, location, rent, amenities, owner details, and lease documents.

- **Lease Model** → Stores lease agreements between tenants and property owners.
- **Payment Model** → Stores rent payment details, transaction status, and Razorpay transaction IDs.
- **Request Model** → Stores tenant rental requests and approval status.

The model layer ensures proper data handling and maintains relationships between users, properties, leases, payments, and requests.

Controller Layer (Business Logic Layer)

The **Controller layer** acts as the core business logic layer of the application. It handles incoming requests, processes user input, interacts with the models, and sends responses back to the client.

The controllers in this project include:

- **Auth Controller** → Handles user registration, login, logout, and authentication.
- **Property Controller** → Handles property creation, updating, deletion, and retrieval.
- **Lease Controller** → Handles lease creation, lease document uploads, and lease management.
- **Payment Controller** → Handles payment processing, payment verification, and transaction records.
- **Request Controller** → Handles tenant rental requests, approvals, and rejections.

The controller layer ensures smooth communication between the frontend and database.

View Layer (Frontend & Routing Layer)

The **View layer** represents the user interface and routing system of the application. It consists of React frontend pages and API routes that define how users interact with the system.

The frontend pages include:

- **Landing Page** → Public homepage for platform introduction.
- **Login Page** → User authentication page.
- **Register Page** → User registration page.
- **Dashboard Page** → User dashboard for quick overview.
- **Properties Page** → Displays available properties.
- **Property Details Page** → Shows detailed property information.
- **Leases Page** → Displays lease details and management.
- **Payments Page** → Displays payment history and tracking.
- **Requests Page** → Displays rental requests and status.

The routing layer handles API requests such as:

- Authentication routes
- Property routes
- Lease routes
- Payment routes
- Request routes

This layer manages user interaction and system navigation.

Advantages of Using MVC in This Project

● **Separation of Concerns:** Each layer has a specific responsibility, improving code organization and maintainability.

● **Scalability:** New modules such as notifications, maintenance requests, or reviews can be added easily.

● **Reusability:** Controllers and models can be reused across different parts of the application.

● **Better Maintenance:** Changes in one layer do not directly affect the others.

● **Improved Security:** Authentication and authorization logic can be managed centrally through middleware.

● **Easy Testing:** Models, controllers, and routes can be tested independently.

● **Team Collaboration:** Frontend and backend developers can work independently on separate layers.

Project Setup and Configuration :

Creating Project Folder

1. Create a new folder with your project name (**Real Estate Rental Management System**).
2. Inside the project folder, create two separate folders.
3. Name one folder as **client**.
4. Name the other folder as **server**.
5. Open the project folder in **VS Code**.

Client Setup (Installing React Application)

- Open the **client** folder in the VS Code terminal and execute the following command:
  - npm create vite@latest . -- --template react
- Select **React** as the framework.
- Select **JavaScript** as the variant.
- Now lets navigate to the client folder by giving the following command.
  - cd client

○ To install all the packages run the following command.

■ npm install ○ To start the React server type the following command.

■ npm run dev

Server Setup (Node.js and Express Setup)

- Open the **server** folder in the VS Code terminal and initialize the backend project:
  - npm init -y
- Install required backend dependencies:
  - npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer razorpay node-cron
- Install Nodemon for development:
  - npm install nodemon --save-dev
- Create the main backend file: server.js
- Create the following folders inside the server directory:
  - models
    controllers
    routes
    middleware
    config
    uploads
    services
- Create the environment configuration file: .env
- Update the package.json file with scripts:

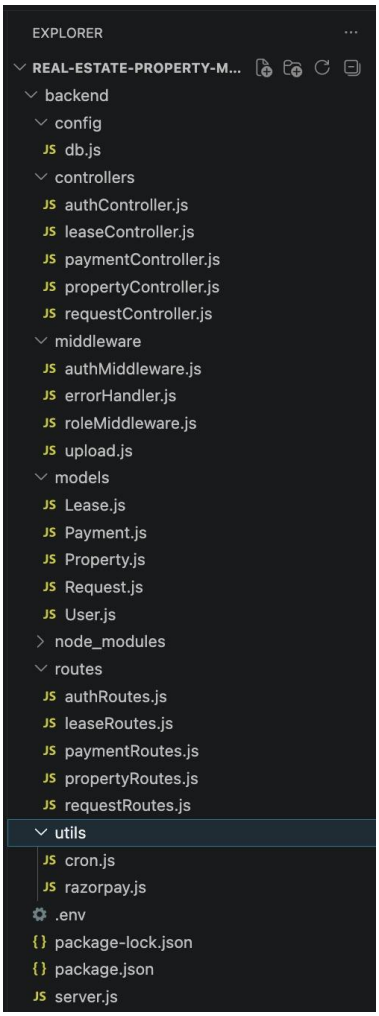
```
"scripts": {
"start": "node server.js",
"dev": "nodemon server.js"
}
```

- Start the backend server:
  - npm run dev

BACKEND DEVELOPMENT:

Backend Reference Video:

Backend Structure :



Controllers (src/controllers)

● authController.js → Handles authentication operations such as user registration, login, logout, and token management.

● propertyController.js → Handles property-related operations such as creating,

updating, deleting, and retrieving property listings.

● leaseController.js → Handles lease agreement creation, management, and document uploads.

● paymentController.js → Handles payment processing, payment verification, and payment history management.

● requestController.js → Handles rental request submission, approval, and rejection processes.

Middleware (src/middleware)

● authMiddleware.js → Middleware for authentication and JWT token validation.

● roleMiddleware.js → Middleware for role-based authorization and access control.

● errorHandler.js → Centralized middleware for handling application errors.

● upload.js → Middleware for handling file uploads such as property images and lease documents.

Models (src/models)

● User.js → Schema/model for storing user account details and roles.

● Property.js → Schema/model for storing property listing information.

● Lease.js → Schema/model for storing lease agreement records.

● Payment.js → Schema/model for storing payment transaction details.

● Request.js → Schema/model for storing tenant rental request details.

Routes (src/routes)

● authRoutes.js → Defines API routes for authentication operations.

● propertyRoutes.js → Defines API routes for property management operations.

● leaseRoutes.js → Defines API routes for lease management operations.

● paymentRoutes.js → Defines API routes for payment processing operations.

● requestRoutes.js → Defines API routes for rental request management operations.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

● Install Mongoose.

In your Node.js project directory, run:

npm install mongoose

● Create database connection.

● Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected");
  } catch (error) {
    console.error(error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

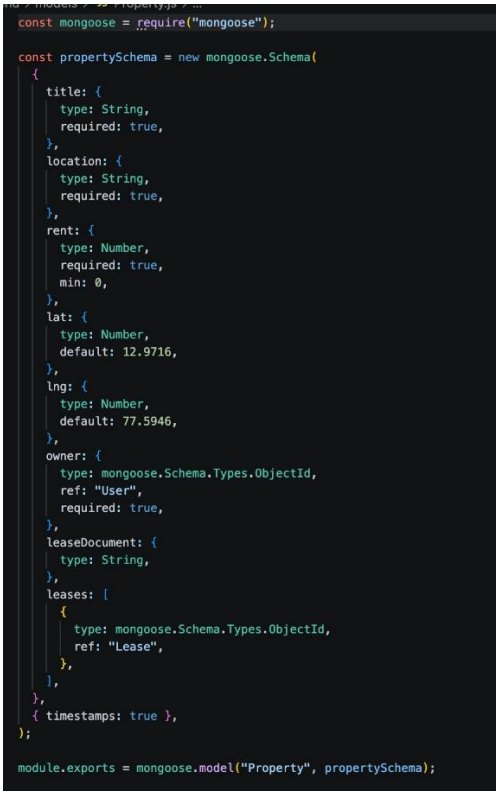
User.js→ Schema/model for storing user account details and roles.

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
    },
    password: {
      type: String,
      required: true,
    },
    role: {
      type: String,
      enum: ["owner", "manager", "tenant"],
      default: "tenant",
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

Request.js→ Schema/model for storing tenant rental request details.

```
const mongoose = require("mongoose");

const requestSchema = new mongoose.Schema(
  {
    tenant: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    property: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Property",
      required: true,
    },
    status: {
      type: String,
      enum: ["pending", "accepted", "rejected"],
      default: "pending",
    },
  },
  { timestamps: true },
);

module.exports = mongoose.model("Request", requestSchema);
```

Property.js→ Schema/model for storing property listing information.



```
const mongoose = require("mongoose");

const propertySchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
    },
    location: {
      type: String,
      required: true,
    },
    rent: {
      type: Number,
      required: true,
      min: 0,
    },
    lat: {
      type: Number,
      default: 12.9716,
    },
    lng: {
      type: Number,
      default: 77.5946,
    },
    owner: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    leaseDocument: {
      type: String,
    },
    leases: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Lease",
      },
    ],
  },
  { timestamps: true },
);

module.exports = mongoose.model("Property", propertySchema);
```

Payment.js→ Schema/model for storing payment transaction details.

```
const mongoose = require("mongoose");

const paymentSchema = new mongoose.Schema(
  {
    tenant: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  property: { type: mongoose.Schema.Types.ObjectId, ref: "Property" },
  amount: { type: Number, required: true },
  paymentDate: { type: Date, default: Date.now },
  status: { type: String, enum: ["paid","pending"], default: "paid" },
}, { timestamps: true });

module.exports = mongoose.model("Payment", paymentSchema);
```

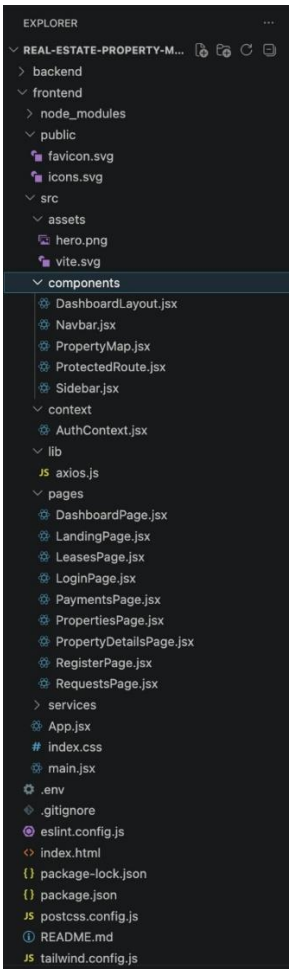
Lease.js→ Schema/model for storing lease agreement records.

```
const mongoose = require("mongoose");

const leaseSchema = new mongoose.Schema(
  {
    tenant: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    property: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Property",
      required: true,
    },
    startDate: {
      type: Date,
      required: true,
    },
    endDate: {
      type: Date,
      required: true,
    },
    document: {
      type: String, // file path
    },
    status: {
      type: String,
      enum: ["active", "completed"],
      default: "active",
    },
  },
  { timestamps: true },
);

module.exports = mongoose.model("Lease", leaseSchema);
```

FRONT-END DEVELOPMENT:

Frontend Reference Video:

Frontend Structure :



Pages (src/pages)

- LandingPage.jsx → Public homepage and entry point of the application.
- LoginPage.jsx → User login and authentication page.

● RegisterPage.jsx → User registration/signup page.

● DashboardPage.jsx → Main dashboard for user overview and quick actions.

● PropertiesPage.jsx → Displays and manages property listings.

● PropertyDetailsPage.jsx → Shows detailed information about a selected property.

● LeasesPage.jsx → Manages lease agreements and lease records.

● PaymentsPage.jsx → Displays payment history and payment tracking details.

● RequestsPage.jsx → Handles rental requests and request management.

Components (src/components)

● DashboardLayout.jsx → Layout wrapper for authenticated dashboard pages.

● Navbar.jsx → Navigation bar used across the application.

● Sidebar.jsx → Sidebar navigation for dashboard pages.

● PropertyMap.jsx → Map component for displaying property locations.

● ProtectedRoute.jsx → Protects routes and restricts unauthorized access.

Context (src/context)

● AuthContext.jsx → Manages authentication state and user session data.

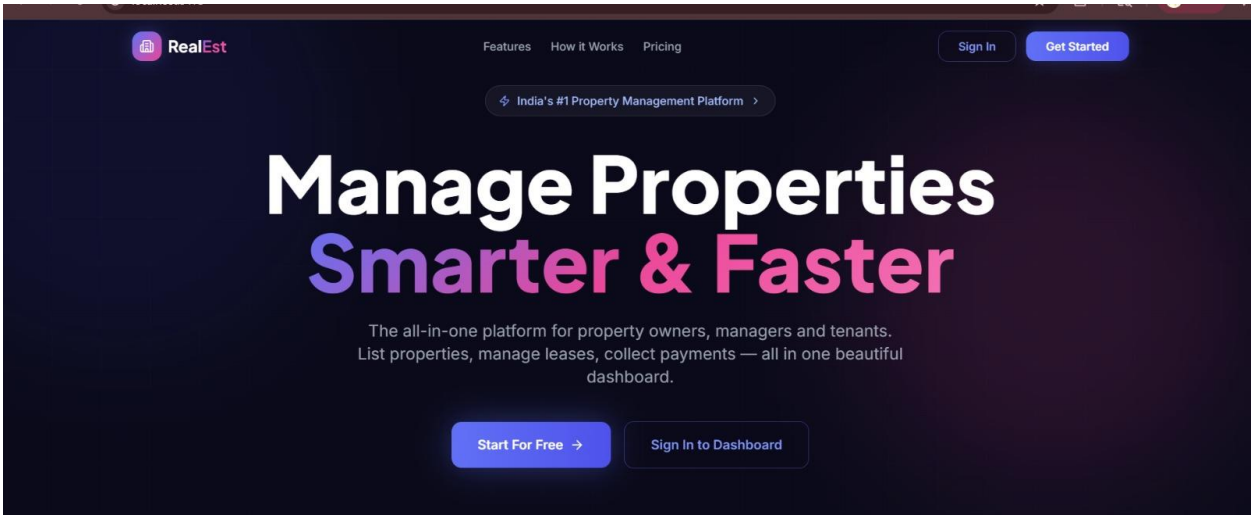
Lib (src/lib)

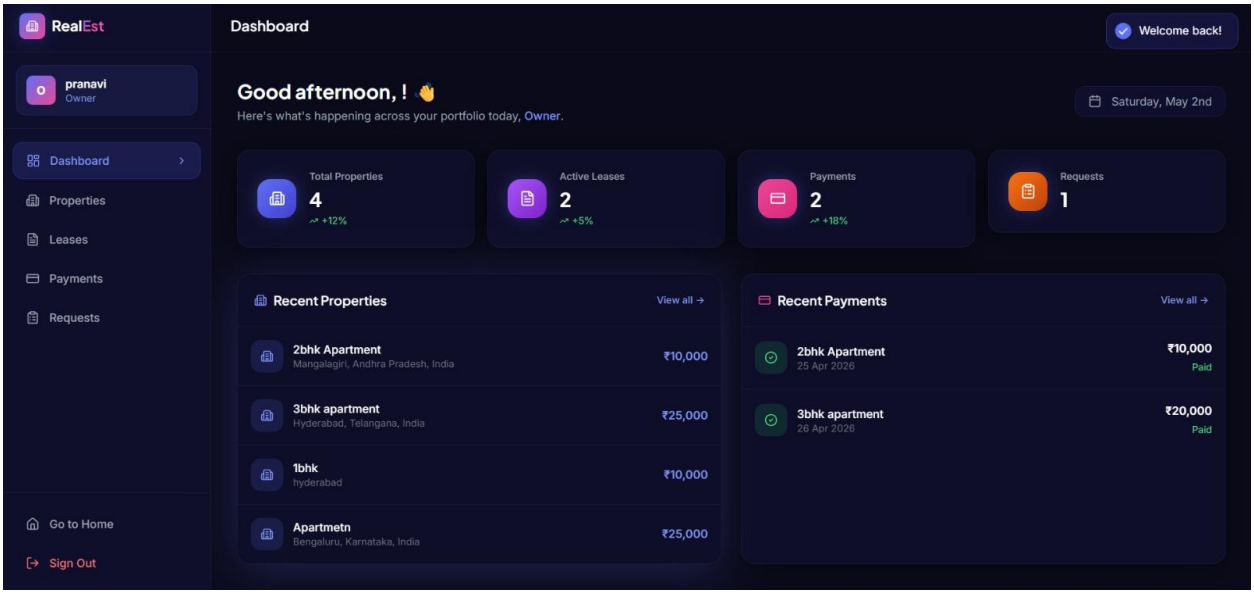
● axios.js → Configured Axios instance for API requests and responses.

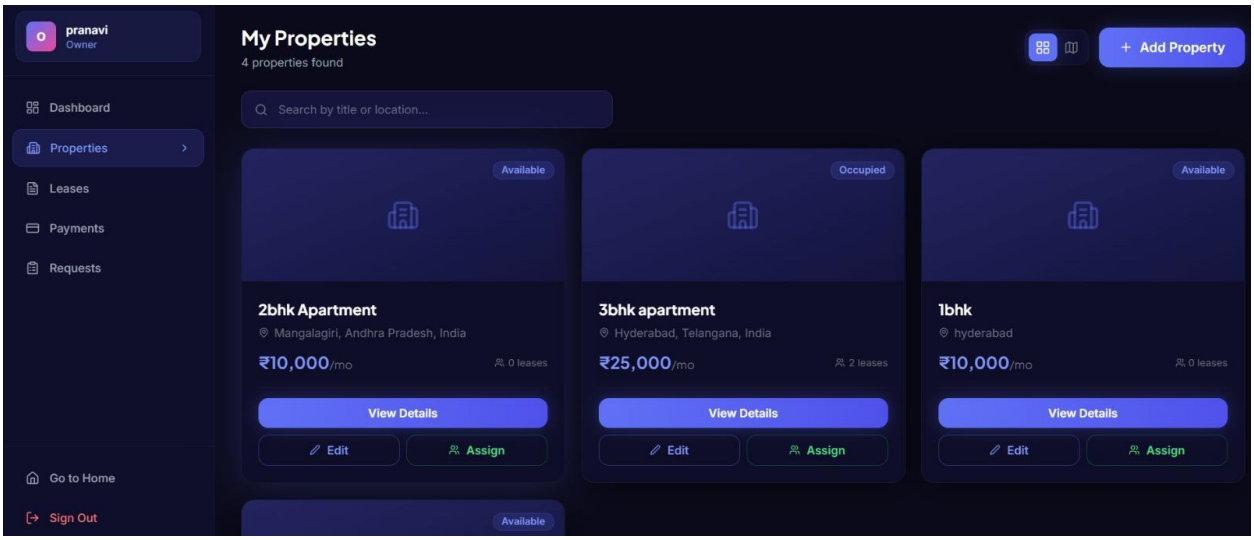
Services (src/services)

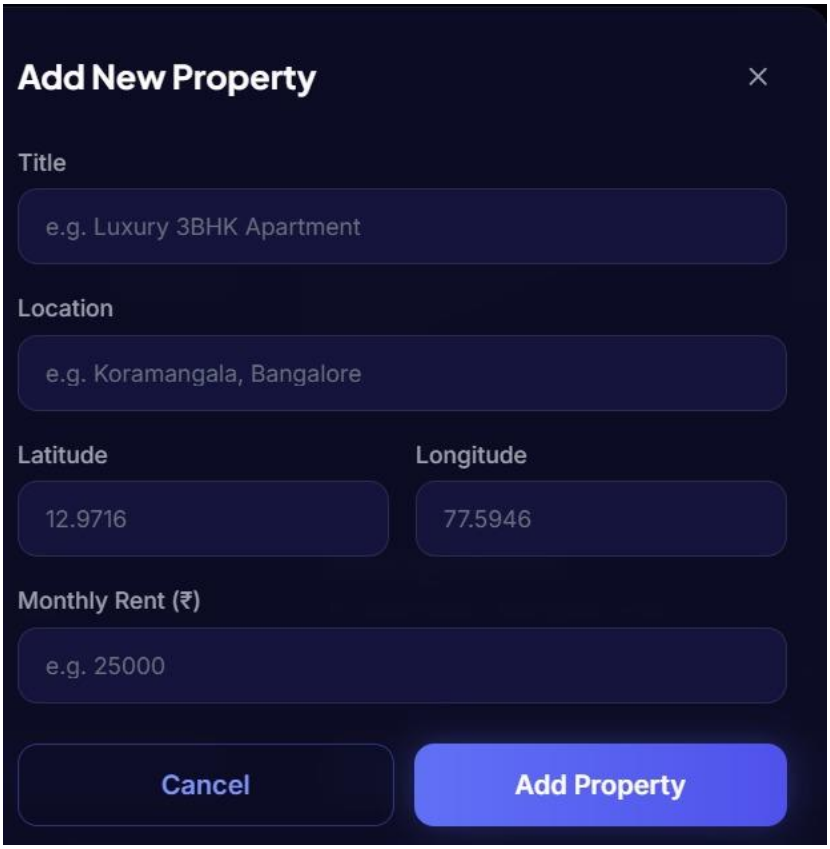
● api.js → Centralized API service functions for backend communication, property management, lease operations, payment processing, and rental request management.

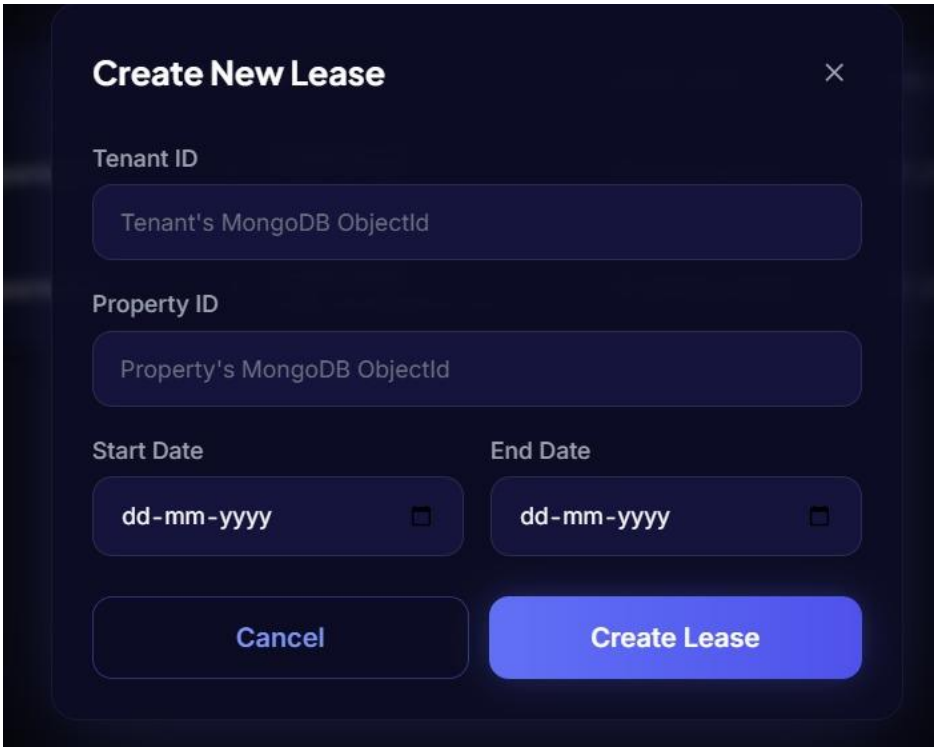
Output ScreenShots:

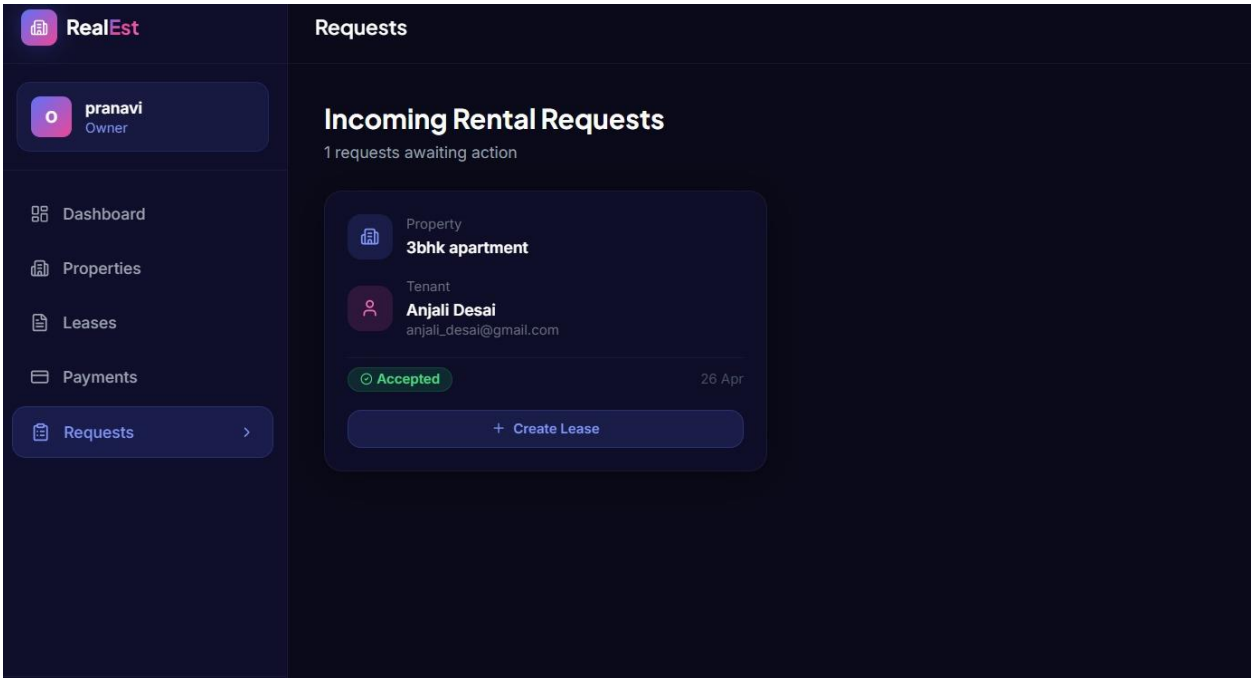


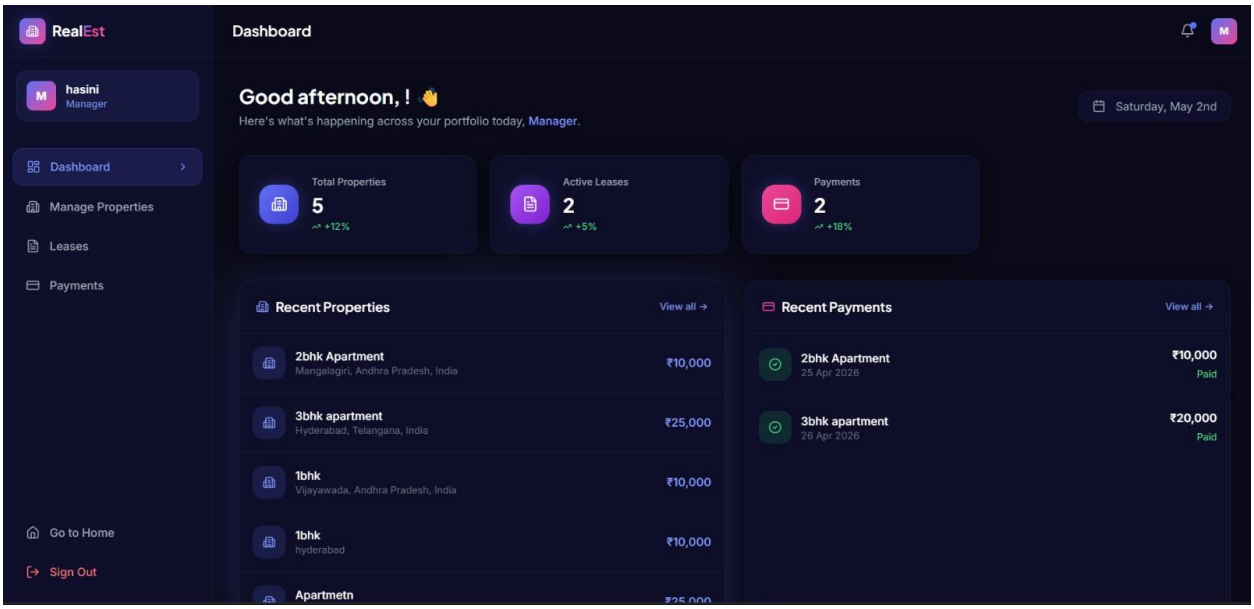


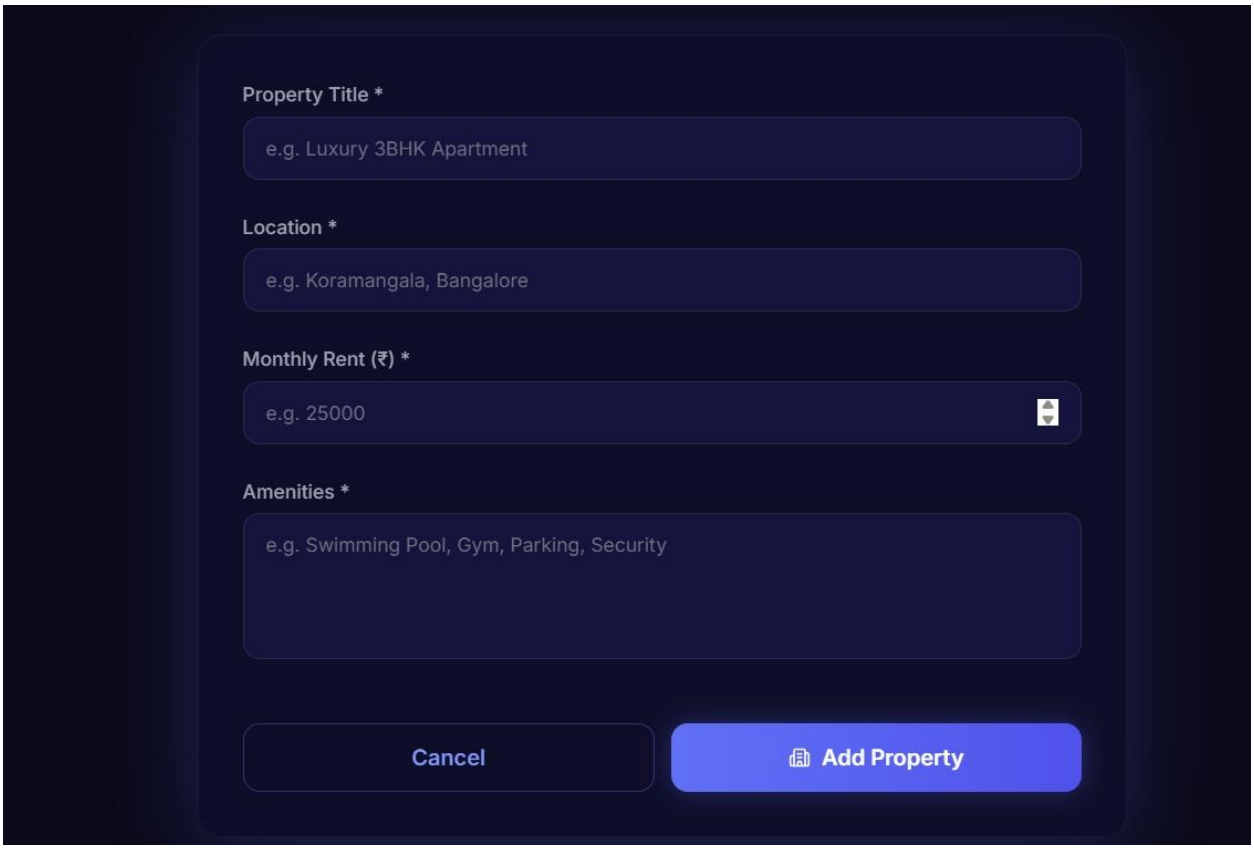

pranavi
Owner
Dashboard
Properties
Leases
Payments
Requests
Go to Home
Sign Out
My Properties
4 properties found
Add Property
Search by title or location...
Available
2bhk Apartment
Mangalagiri, Andhra Pradesh, India
₹10,000/mo
0 leases
View Details
Edit
Assign
Occupied
3bhk apartment
Hyderabad, Telangana, India
₹25,000/mo
2 leases
View Details
Edit
Assign
Available
1bhk
hyderabad
₹10,000/mo
0 leases
View Details
Edit
Assign
Available

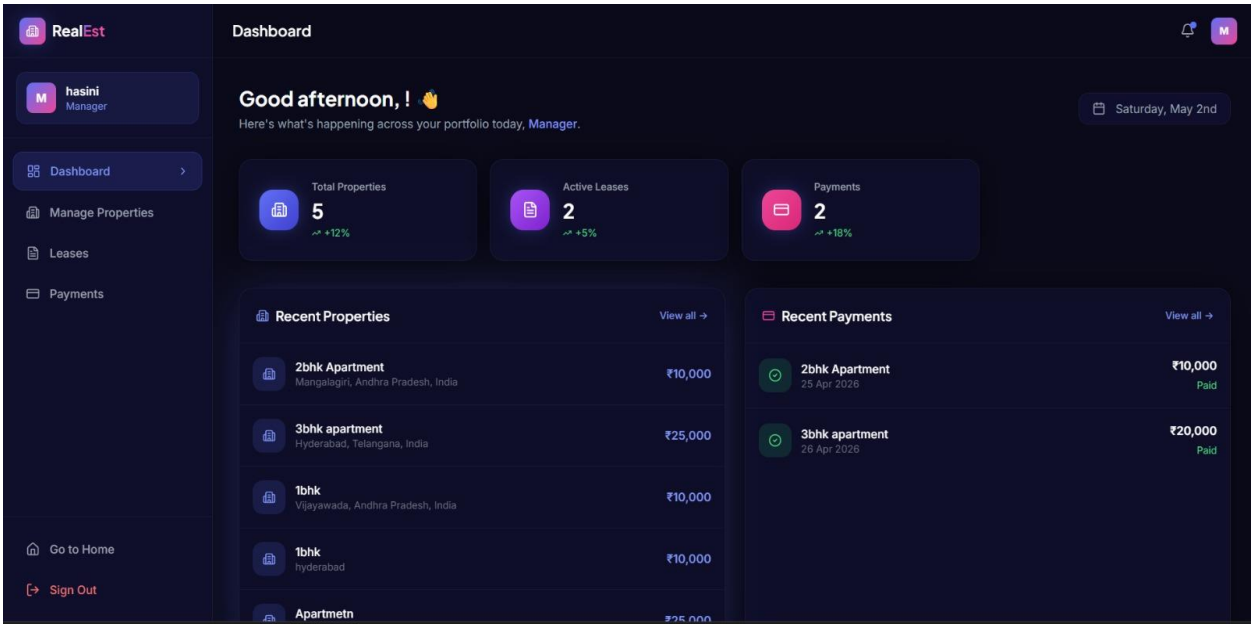

Add New Property
Title
e.g. Luxury 3BHK Apartment
Location
e.g. Koramangala, Bangalore
Latitude
12.9716
Longitude
77.5946
Monthly Rent (₹)
e.g. 25000
Cancel
Add Property

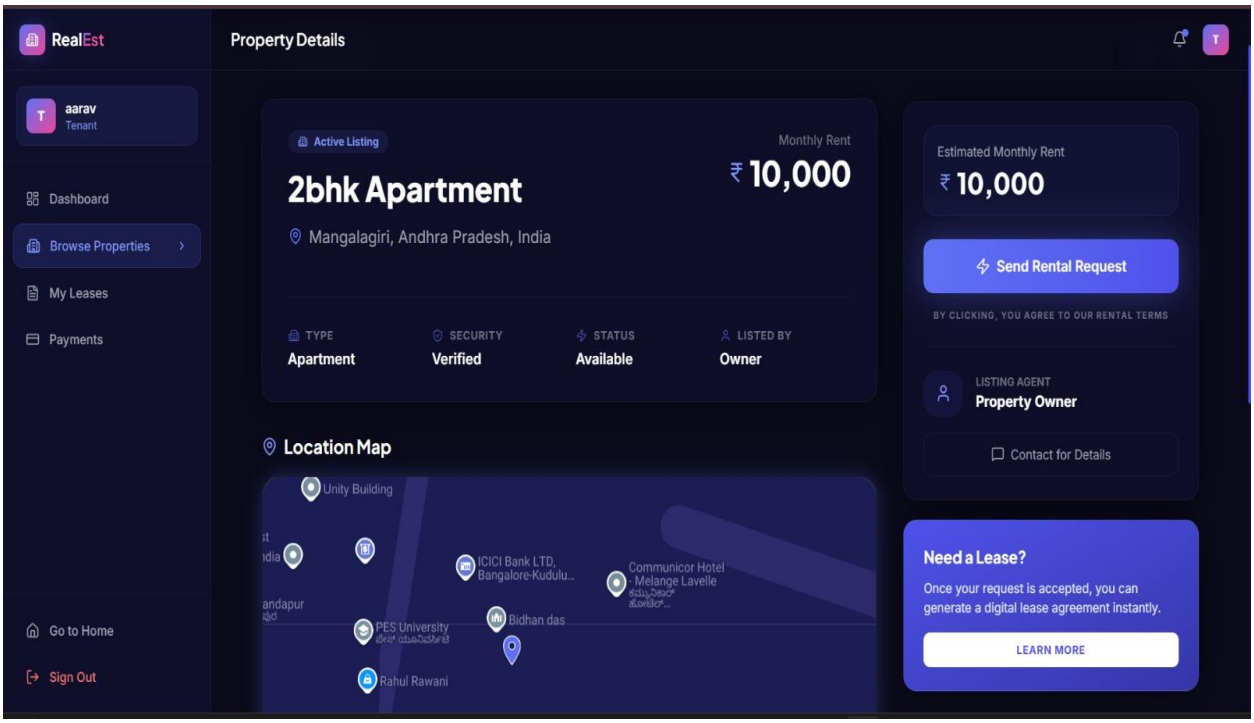

Create New Lease
Tenant ID
Tenant's MongoDB ObjectId
Property ID
Property's MongoDB ObjectId
Start Date
dd-mm-yyyy
End Date
dd-mm-yyyy
Cancel
Create Lease

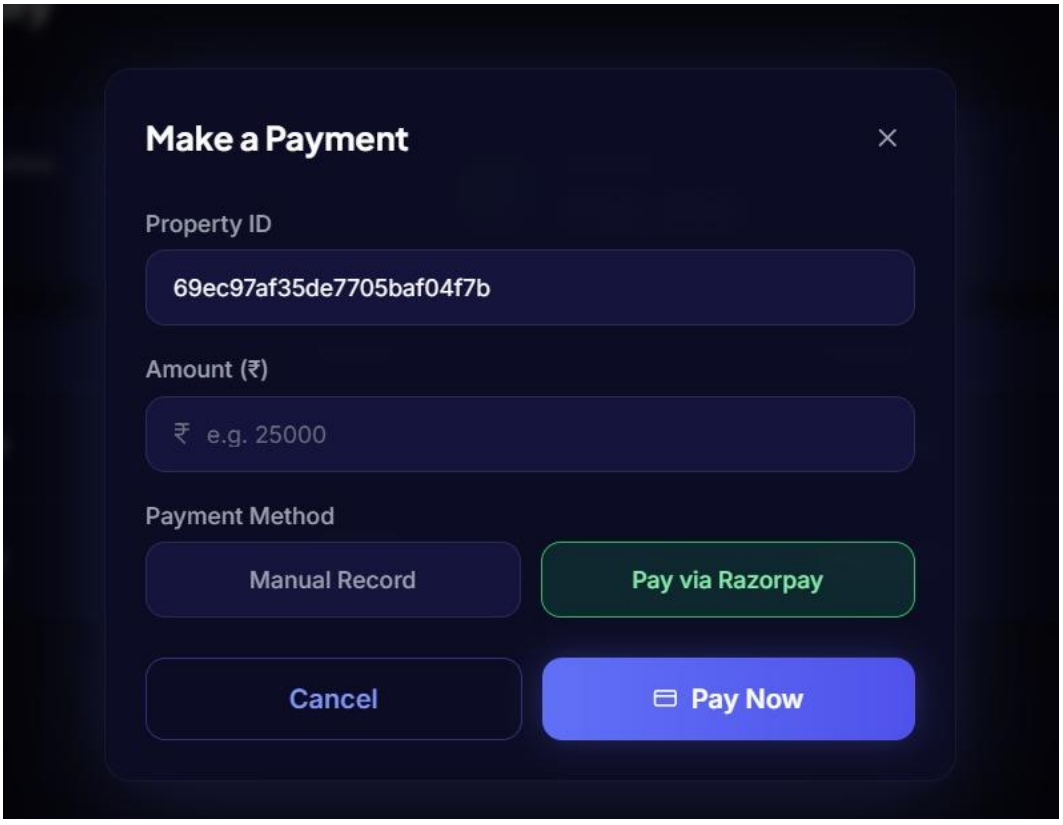

RealEst
pranavi
Owner
Dashboard
Properties
Leases
Payments
Requests
Requests
Incoming Rental Requests
1 requests awaiting action
Property
3bhk apartment
Tenant
Anjali Desai
anjali_desai@gmail.com
Accepted
26 Apr
Create Lease

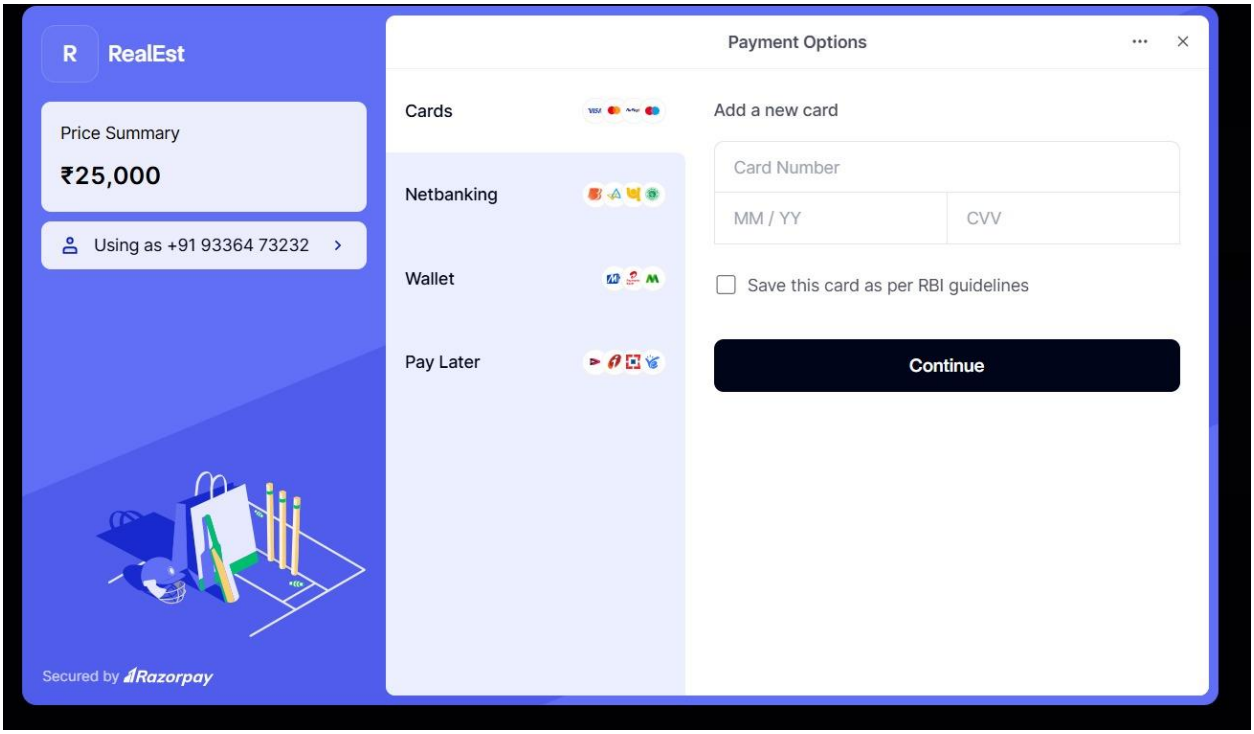

RealEst
Dashboard
hasini
Manager
Dashboard
Manage Properties
Leases
Payments
Go to Home
Sign Out
Good afternoon, ! 👋
Here's what's happening across your portfolio today, Manager.
Saturday, May 2nd
Total Properties
5
+12%
Active Leases
2
+5%
Payments
2
+18%
Recent Properties
View all →
2bhk Apartment
Mangalagiri, Andhra Pradesh, India
₹10,000
3bhk apartment
Hyderabad, Telangana, India
₹25,000
1bhk
Vijayawada, Andhra Pradesh, India
₹10,000
1bhk
hyderabad
₹10,000
Apartmetn
Recent Payments
View all →
2bhk Apartment
25 Apr 2026
₹10,000
Paid
3bhk apartment
26 Apr 2026
₹20,000
Paid

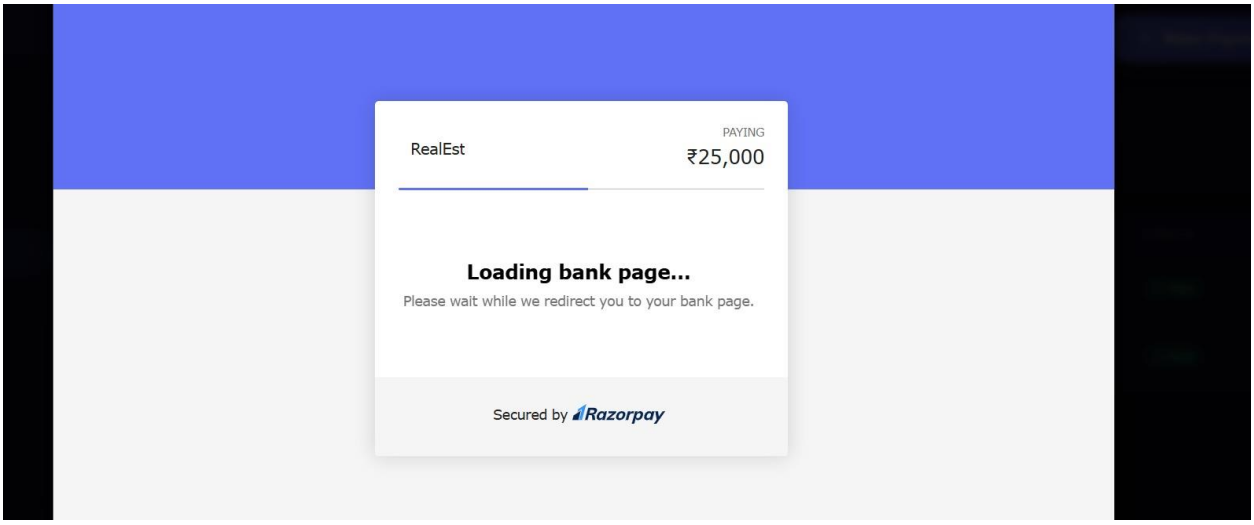

Property Title *
e.g. Luxury 3BHK Apartment
Location *
e.g. Koramangala, Bangalore
Monthly Rent (₹) *
e.g. 25000
Amenities *
e.g. Swimming Pool, Gym, Parking, Security
Cancel
Add Property

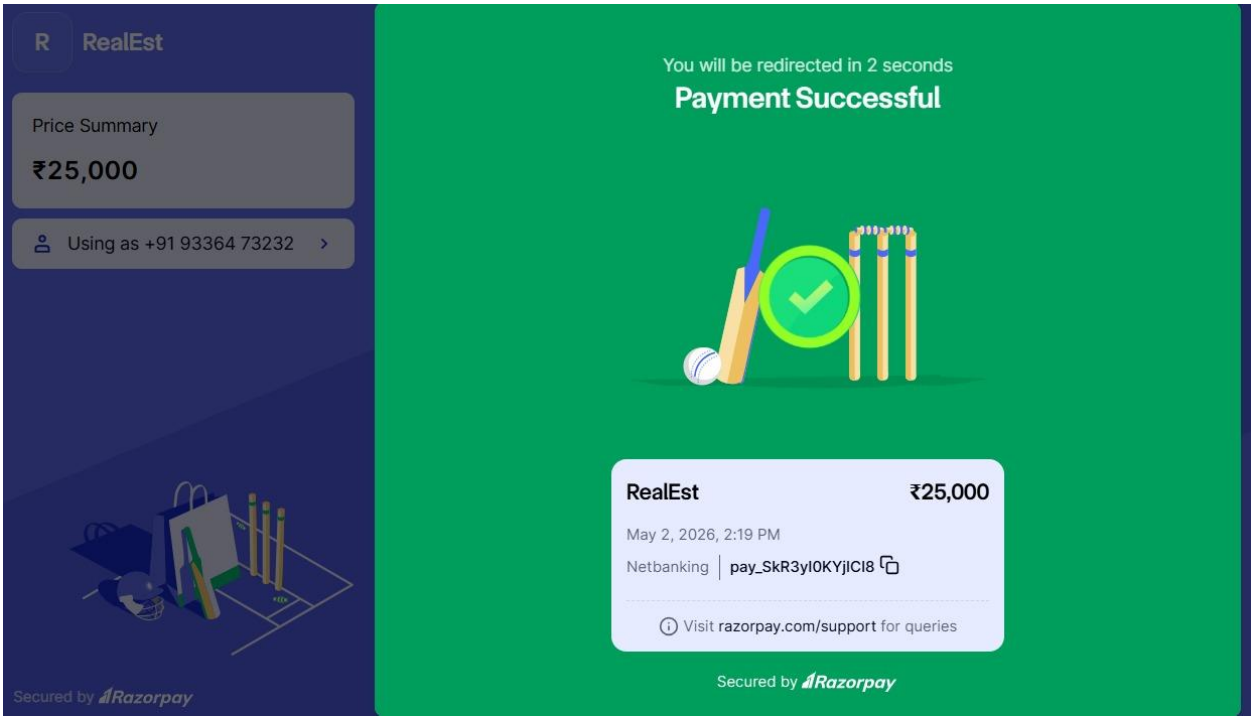

RealEst
Dashboard
M
hasini
Manager
Dashboard
Manage Properties
Leases
Payments
Go to Home
Sign Out
Dashboard
Good afternoon, ! 👋
Here's what's happening across your portfolio today, Manager.
Saturday, May 2nd
Total Properties
5
+12%
Active Leases
2
+5%
Payments
2
+18%
Recent Properties
View all →
2bhk Apartment
Mangalagiri, Andhra Pradesh, India
₹10,000
3bhk apartment
Hyderabad, Telangana, India
₹25,000
1bhk
Vijayawada, Andhra Pradesh, India
₹10,000
1bhk
hyderabad
₹10,000
Apartmetn
Recent Payments
View all →
2bhk Apartment
25 Apr 2026
₹10,000
Paid
3bhk apartment
26 Apr 2026
₹20,000
Paid


RealEst
Property Details
T
aarav
Tenant
Dashboard
Browse Properties
My Leases
Payments
Go to Home
Sign Out
Active Listing
2bhk Apartment
Mangalagiri, Andhra Pradesh, India
Monthly Rent
₹ 10,000
TYPE
Apartment
SECURITY
Verified
STATUS
Available
LISTED BY
Owner
Location Map
Unity Building
ICICI Bank LTD, Bangalore-Kudulu...
Communicor Hotel - Melange Lavelle
PES University
Bidhan das
Rahul Rawani
Estimated Monthly Rent
₹ 10,000
Send Rental Request
BY CLICKING, YOU AGREE TO OUR RENTAL TERMS
LISTING AGENT
Property Owner
Contact for Details
Need a Lease?
Once your request is accepted, you can generate a digital lease agreement instantly.
LEARN MORE


Make a Payment
Property ID
69ec97af35de7705baf04f7b
Amount (₹)
₹ e.g. 25000
Payment Method
Manual Record
Pay via Razorpay
Cancel
Pay Now


R
RealEst
Price Summary
₹25,000
Using as +91 93364 73232
Secured by Razorpay
Payment Options
Cards
Netbanking
Wallet
Pay Later
Add a new card
Card Number
MM / YY
CVV
Save this card as per RBI guidelines
Continue


RealEst
PAYING
₹25,000
Loading bank page...
Please wait while we redirect you to your bank page.
Secured by Razorpay


R
RealEst
Price Summary
₹25,000
Using as +91 93364 73232
Secured by Razorpay
You will be redirected in 2 seconds
Payment Successful
RealEst
₹25,000
May 2, 2026, 2:19 PM
Netbanking
pay_SkR3yl0KYjICI8
Visit razorpay.com/support for queries
Secured by Razorpay